

Lab 4

Title: Design and implementation of Push down Automata.

Theory:

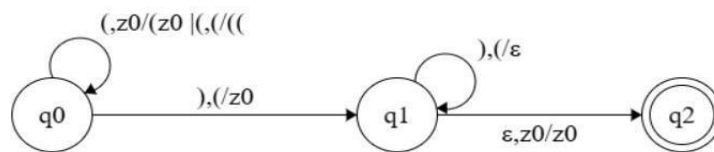
A Pushdown Automaton (PDA) is a computational model that extends the capabilities of a Finite Automaton by adding a stack. This stack acts as an auxiliary memory, allowing the PDA to recognize a broader class of languages known as context-free languages. Unlike the finite memory of a DFA or NFA, the stack provides an unlimited, last in, first-out (LIFO) memory, which is essential for handling nested structures like balanced parentheses or nested loops.

Components of a PDA

A PDA is formally defined as a 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where:

- Q is a finite set of states.
- Σ is a finite set of input symbols (the alphabet).
- Γ (Gamma) is a finite set of stack symbols.
- δ (Delta) is the transition function. It maps a state, an input symbol (or ϵ), and a stack symbol to a new state and a string of stack symbols to be pushed. Formally, $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2Q \times \Gamma^*$.
- q_0 is the start state.
- Z_0 is the initial stack symbol, an element of Γ . This symbol is on the stack when the PDA begins.
- F is a set of final (or accepting) states.

Program 1: Design and implementation of PDA for Balanced Parentheses.



Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#define MAX 1000

int main()
{
    char str[MAX];
    char stack[MAX];
    int top = -1;
    printf("Program by: Mahesh Udas\n");
    printf("Enter parentheses string: ");
    scanf("%s", str);
    for (int i = 0; i < strlen(str); i++)
    {
        if (str[i] == '(')
        {
            stack[++top] = '(';
        }
        else if (str[i] == ')')
        {
            if (top == -1)
            {
                printf("Rejected: Not balanced\n");
                return 0;
            }
            top--;
        }
    }
    if (top == -1)
        printf("Accepted: Balanced parentheses\n");
    else
        printf("Rejected: Not balanced\n");
    return 0;
}
```

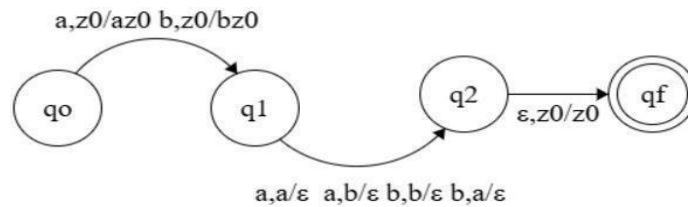
Output and discussion

```
Program by: Mahesh Udas
Enter parentheses string: ()
Accepted: Balanced parentheses
```

```
Program by: Mahesh Udas
Enter parentheses string: ())
Rejected: Not balanced
```

This C program checks whether a string of parentheses is balanced using a stack-based approach. It pushes each opening bracket '(' onto the stack and pops for each closing bracket ')'. If a closing bracket appears without a matching opening, or if any unmatched opening brackets remain after processing, the string is rejected. Otherwise, it is accepted. The program demonstrates fundamental stack operations and validates syntactic correctness of parentheses, which is essential in parsing expressions, compiler design, and evaluating mathematical formulas. It is a concise and effective implementation of bracket matching logic using arrays and control structures.

Program 2: Design and implementation of PDA for even length strings over $\{a,b\}$.



Source code:

```

#include <stdio.h>
#include <string.h>
int main()
{
    char str[1000];
    int state = 0;

    printf("Program by: Mahesh Udas\n");
    printf("Enter string over {a,b}: ");
    scanf("%s", str);
    for (int i = 0; i < strlen(str); i++)
    {
        if (str[i] == 'a' || str[i] == 'b')
        {
            state = 1 - state;
        }
        else
        {
            printf("Invalid character detected!\n");
            return 0;
        }
    }
    if (state == 0)
        printf("Accepted: String has even length.\n");
    else
        printf("Rejected: String has odd length.\n");
    return 0;
}
  
```

Output and discussion

```

Program by: Mahesh Udas
Enter string over {a,b}: abaaa
Rejected: String has odd length.
  
```

```

Program by: Mahesh Udas
Enter string over {a,b}: abab
Accepted: String has even length.
  
```

This C program checks whether a string over the alphabet $\{a, b\}$ has even length using a simple state-flipping mechanism. It starts in state 0 and toggles between states 0 and 1 for each valid character. If any character is not 'a' or 'b', it flags an error. After processing the entire string, it accepts if the final state is 0 (even length), and rejects if the state is 1 (odd length). This approach cleverly uses state transitions to determine string length parity without explicitly counting characters, making it an elegant demonstration of finite automata principles in action.

Conclusion:

This lab successfully demonstrated the design and implementation of Pushdown Automata (PDA) for recognizing context-free languages. Two programs were developed: one for validating balanced parentheses using stack operations, and another for accepting even-length strings over $\{a,b\}$ using state transitions. The balanced parentheses program effectively utilized LIFO stack mechanics to handle nested structures, pushing opening brackets and popping for closing ones. The even-length string validator employed elegant state-flipping logic to determine length parity without explicit counting. These implementations showcased how PDAs extend finite automata capabilities through auxiliary memory, enabling recognition of more complex language patterns essential for parsing, compiler design, and syntactic analysis in programming language.